Mydrak AI v3

Final Master Specification

The plant doctor that knows when it doesn't know.

Meta Global AI Developer Hackathon 2026 | Track: Indie Developer | Theme 1: Build for Your World

---

How to use this document. This is the single source of truth for the project. A human or a coding agent (Muse Code) should read it top to bottom and understand what is being built, why, how it fits together, what order to build it in, and what rules to follow. Part A is the big picture. Part B is the system design. Part C is the build plan and agent rules. Part D is evaluation, demo and submission. Part E is the refined strategy that separates a top-5 finish from a good submission.

What changed from v2: Scope reduced to a defensible core. Eval moved earlier. KB narrowed. Demo focused on three killer moments. Added Part E (Top-5 Finish Playbook). Added explicit MVP Core definition. Added cost/accuracy chart as the pitch centerpiece.

DOCUMENT MAP

PART A — THE BIG PICTURE
Problem, product vision, MVP Core, routing strategy and principles

PART B — SYSTEM DESIGN
Architecture, models, Decider, severity, knowledge base, data and safety

PART C — BUILD PLAN AND AGENT RULES
Roadmap, task backlog and implementation rules

PART D — EVALUATION, DEMO AND SUBMISSION
Evaluation methodology, demo, README and submission readiness

PART E — TOP-5 FINISH PLAYBOOK
Pitch, judge Q&A, demo resilience and final 48 hours

How to use this document. This is the single source of truth for the project. A human or a coding agent (Muse Code) should read it top to bottom and understand what is being built, why, how it fits together, what order to build it in, and what rules to follow. Part A is the big picture. Part B is the system design. Part C is the build plan and agent rules. Part D is evaluation, demo and submission. Part E is the refined strategy that separates a top-5 finish from a good submission.

What changed from v2: Scope reduced to a defensible core. Eval moved earlier. KB narrowed. Demo focused on three killer moments. Added Part E (Top-5 Finish Playbook). Added explicit MVP Core definition. Added cost/accuracy chart as the pitch centerpiece.

PART A — THE BIG PICTURE

A1. Hackathon facts (locked)

Item	Value
Event	Meta Global AI Developer Hackathon 2026
Track and theme	Indie Developer. Theme 1: Build for Your World
Theme text	Bring Meta's models into your domain. Solve a problem you know firsthand, free up time and mental space, and make something possible that wasn't before
Kickoff	Oct 26 (Day 0)
Submission deadline	Nov 4, 9:00 PM ET = Nov 5, 6:30 AM IST
Personal submit target	Nov 4, 12:00 PM IST (about 18 hours early)
Live finals	Nov 9. Top 5 per track present live
Build window	10 days, Oct 26 to Nov 4
Meta stack	Muse Spark 1.3, Muse Glimmer, SAM 3.1, Muse Voice Transcribe, Muse Code (builds the project), Muse Image (stretch only)
Total prize pool	$1,000,000 across Indie + Startup tracks
Indie track pool	$400,000 (1st: $150,000)

Judging process (three stages)

Stage 1, pass/fail: does the project fit a theme and use Meta AI products? Mydrak must make both obvious in the first three lines of the README and the first 10 seconds of the video.

Stage 2, rubric scoring: judges score every passing project 0–10 on each criterion.

Stage 3, live final: top 5 per track. Judges watch demos live on Nov 9 and rank them using the same rubric.

Rubric

Criterion	Weight	What judges look for	Mydrak's answer
Impact	30%	Real-world potential, who benefits, fit with a problem statement	Smallholder farmers with no access to agronomists; Hindi voice-first; honest uncertainty; open-weight self-host option for a KVK or NGO
Use of Meta AI tooling	25%	Creative use beyond a basic integration	Five Meta products with distinct jobs, router across two Meta model tiers, SAM-based measurement, Muse Code as the builder with logs as proof
Technical depth	20%	Sound, refined engineering with real craft	Multi-signal Decider, accuracy-vs-coverage evaluation, KB-grounded verification, bounded retries, tracing and cost accounting
Demo	25%	A working demo that holds up, and a demo video worth watching	3-minute story, cached replay mode, recorded backup video, one cost/accuracy chart

Implication: a flawless demo of a smaller feature set beats a broken demo of a larger one. Impact is the largest weight, so the story of the farmer matters as much as the architecture.

Official Rules (must read on Day 0)

The Official Rules are published on kickoff and must be accepted. Read them for: submission format, demo video length, whether non-Meta tools (Telegram, hosting, database, ffmpeg) are allowed, team and licensing terms, and any requirement to prove use of Muse Code. Record answers in Section D6.

A2. The problem

Small-scale farmers often cannot identify crop diseases early or get reliable guidance. The barriers are:

Limited access to agricultural experts and extension services.

Language barriers: most guidance is written in English or technical language.

Low digital literacy: typing queries or using complex apps is hard; voice and photos are natural.

Cost of professional consultations.

Delay: waiting days for advice can mean avoidable crop loss, or wrong and costly pesticide use.

Who benefits: smallholder farmers (first), extension workers and KVK staff (triage support), NGOs and farmer groups (self-hostable tool).

What is new: plant-disease classifiers already exist, so a label alone is not the point. Mydrak's value is a complete, honest workflow: measure how bad it is, say how sure it is, ask when unsure, ground advice in a verified knowledge base, track recovery, and do all of it cheaply by routing between models.

A3. Product vision

Mydrak AI is an image-first, voice-friendly crop health assistant on Telegram. A farmer sends a photo of an affected plant and optionally a voice note. Mydrak replies in Hindi or English with:

The likely disease or crop health issue.

Visual evidence on the farmer's own photo (lesion overlay).

Severity (percent of leaf affected) in plain words.

Confidence or uncertainty in plain words, never a fake-precise number.

Practical, low-cost, low-risk next steps from a verified knowledge base.

A safety line and, when needed, guidance to consult a qualified agricultural expert (Kisan Call Centre or a local KVK).

Core principle: Mydrak never presents an uncertain prediction as a confirmed diagnosis. It gives accessible preliminary guidance and does not replace professional agricultural expertise.

The five differentiators

Severity from SAM, not just a label. Spark decides what to segment (leaf, lesions). SAM 3.1 returns masks. Code computes percent of leaf area affected. The farmer gets their photo back with lesions highlighted.

Diagnose by asking. When evidence is ambiguous, ask one or two targeted follow-up questions instead of guessing.

Recovery tracking. A re-sent photo days later is compared by lesion area: spreading, no clear change, or improving.

Hindi voice-first. Voice note in, Hindi reply out, English as second language.

Honest refusal as a feature. If uncertain after one escalation, say so and refer to an expert. Treatment advice comes only from the curated knowledge base, never from model memory, because pesticide advice is a safety issue.

Central innovation (for the pitch)

The innovation is not "AI recognizes plant disease." It is a system that makes preliminary agricultural guidance more accessible and affordable while accounting for model limitations, uncertainty and operating cost: cost-aware routing, measurable severity, knowledge verification, and honest escalation.

One-sentence pitch: "Mydrak matches the accuracy of Meta's strongest model at a fraction of the cost, by knowing when to escalate — and knows when to say 'I cannot tell.'"

A4. The Insane MVP Core (explicitly defined)

This is the single most important section. If you build only this, you have a top-5 project.

A farmer sends a Hindi voice note + photo. Mydrak replies with their own photo, lesions highlighted, severity as a percentage, a plain-word confidence ("fairly sure" / "not sure" / "cannot tell"), KB-grounded steps, and a safety line. If it's unsure, it asks one question. If it's still unsure, it refuses and refers. The dashboard shows it matched Spark's accuracy at ~40% of the cost.

The MVP Core is the demo. Everything else is amplification.

Layer	In MVP Core?	Why
Telegram bot with photo input	[YES] Yes	Entry point
Hindi voice note → transcription → reply	[YES] Yes	The accessibility claim must be demonstrated, not described
SAM overlay + severity %	[YES] Yes	The visual "wow" moment
Glimmer tier 1 + Decider + one Spark escalation	[YES] Yes	The central innovation
KB-grounded steps + safety line	[YES] Yes	Safety and credibility
Honest refusal path (ask → refer)	[YES] Yes	The killer differentiator
Dashboard with cost/accuracy chart	[YES] Yes	The technical-depth proof point
Eval harness (30 images, 3 strategies)	[YES] Yes	Every number in the pitch must come from here
Replay mode + backup video	[YES] Yes	Demo reliability
Recovery tracking	[SHOULD-HAVE] Should-have	Adds a beat if time permits
Follow-up questions beyond one turn	[SHOULD-HAVE] Should-have	One question is enough for the demo
Muse Image pictograms	[NO] Stretch	Never touch unless everything is green
Multi-language beyond Hindi/English	[NO] Stretch	Out of scope for 10 days

If you are behind schedule, cut from the bottom of this table. Never cut from the top.

A5. What changed from v2 (traceability)

v2 Component	Status in v3	Reason
50-image eval, labeled on Day 6	Moved to 30 images, labeled Day 1–2	Eval is 20% of technical depth; cannot be a Day 6 scramble
20 KB cards, 4–5 crops	Narrowed to 12–16 cards, 3 crops done perfectly	Better 3 crops with deep coverage than 5 with thin cards
8-beat demo script	Compressed to 3 killer moments + supporting beats	Judges remember three things, not eight
Hindi review on Day 8	Moved to Day 2–3, final check Day 8	Credibility issue if Hindi is awkward; fix early
SAM demo images selected ad hoc	Pre-selected 5–10 clean images, tested Day 3	Demo-killer if masks are poor on the day
Recovery tracking in MVP	Moved to Should-Have	Adds a beat but not core to the thesis
Generic "future directions"	Added Part E: Top-5 Finish Playbook	Execution strategy is now explicit
Cost/accuracy chart as one of several eval outputs	Promoted to pitch centerpiece	Single most powerful proof point

A6. Cost-aware routing strategy (central innovation, in detail)

Instead of sending every request to the largest model, Mydrak tries a lower-cost model first and escalates only when evidence says it should.

Tier 1: Muse Glimmer (open 30B, Apache 2.0, multimodal, single-GPU class). Cheap. Can be self-hosted by a KVK or NGO. Sampled twice per case for agreement.

Tier 2: Muse Spark 1.3 (multimodal, tool calling). Stronger reasoning. Used once per case at most, with SAM crops as extra evidence.

Note: Spark Open is 816B parameters and not locally runnable. Glimmer at 30B on a laptop is doubtful. The system must not depend on local model inference; use a hosted provider for Glimmer if needed (Fireworks, Together or OpenRouter, to be verified Day 0).

COST_MODE setting

COST_MODE	Meaning
local (default)	Minimum cost: local SQLite and file storage, hosted API calls only where needed, no paid cloud account required, aggressive caching
production	Optional cloud storage and logging, higher limits, more samples allowed

Cost-control measures

Image resizing and preprocessing (cap long edge, JPEG recompress) before any model call.

Caching of repeated requests (hash of preprocessed image plus language plus voice hint).

Request and output limits (max tokens per call, max follow-up turns).

Bounded retries (at most 2 per call) and a single controlled escalation per case.

Monitoring of model usage, latency and estimated cost per case.

Truth in claims: actual costs and free-tier eligibility depend on hardware, licenses, API quotas and hosting. Verify before quoting any cost figure.

A7. Non-goals (what Mydrak will not do)

It will not give a confirmed diagnosis, dosage, or pesticide instruction from model memory.

It will not claim general reliability across crops, regions and seasons from a 30-image test.

It will not support WhatsApp, AWS deployment, or more than 3 crops in the MVP.

It will not store raw farmer data beyond the configured retention window.

It will not generate Hindi treatment text live; all Hindi text is pre-written and reviewed.

A8. Principles that resolve design arguments

Honesty beats coverage: refer or ask rather than guess.

Safety beats helpfulness for anything touching chemicals.

Demo reliability beats feature count.

Measured beats claimed: every number in the pitch comes from the eval harness.

Cheap first, escalate once.

Simple beats clever: plain loops, pure functions, small modules.

Build the MVP Core first; amplify only if green.

A9. Scope: MVP, should-have, stretch (v3)

Tier	Item
MVP Core	Telegram bot with photo plus Hindi voice note
MVP Core	Hindi and English replies: crop and issue, plain-word confidence, SAM overlay, severity %, KB-grounded steps, safety line, expert referral rule
MVP Core	Decider routing with one escalation and bounded retries
MVP Core	One fully tested Hindi voice flow
MVP Core	Dashboard: per-case trace, latency, cost, route taken

Tier	Item
MVP Core	Eval harness: 30 labeled images, 3 strategies, accuracy vs coverage, cost/accuracy chart
MVP Core	Demo replay mode and recorded backup video
Should have	Follow-up questions (one turn is enough for demo; two if time)
Should have	Recovery re-scan (adds a demo beat if green by Day 7)
Stretch	More languages, Muse Image pictogram cards, text-to-speech replies, optional cloud deployment

A10. Future directions (post-hackathon, for the README and pitch)

Region-specific crop guidance, expanded disease coverage, crop health history per farmer, preventive care recommendations, connections to agricultural experts and local support services, more languages, text-to-speech replies, pictogram cards (Muse Image) for low-literacy users, WhatsApp support, and self-hosted deployment kits for KVKs and NGOs. With appropriate validation and responsible deployment, Mydrak could grow into a broader digital agricultural assistance platform.

Mission: help farmers make more informed crop health decisions through accessible AI, responsible guidance, and intelligent use of computational resources.

PART B — SYSTEM DESIGN

B1. Architecture overview

```
Farmer (Telegram)
   | photo (+ optional voice OGG)
   v
Bot handler -- voice --> ffmpeg (OGG->wav 16k mono) --> Voice Transcribe --> text hint
   |
   v
Preprocess: EXIF fix, resize, quality check
   |  (quality fail -> ask for a better photo, stop)
   v
Cache lookup (hash) -- hit --> reuse stored result
   |
   v
Tier 1: Glimmer x2 samples -> structured JSON (crop, issue, symptoms, confidence)
   |
   v
Decider: S1 agreement + S2 KB match + S3 quality + S4 stated confidence -> score
   |-- score >= accept ---------------------------------+
   |-- escalate band / disagreement:                    |
   |      Spark 1.3 (+ SAM crops) once -> re-score      |
   |-- still unsure: follow-up question (max 2),        |
   |      then expert referral                          |
   v                                                    v
SAM 3.1: leaf + lesion masks -> severity % + overlay <---+
   v
KB lookup (crop + disease card) -> steps from KB only
   v
Reply composer (Hindi / English templates) + overlay image -> Telegram
   v
Trace row -> SQLite -> Dashboard
Module responsibilities
Module■Responsibility■Never does
bot■Telegram updates, conversation state, sending replies■Model calls, business logic
audio■OGG to wav, transcription, language hint■Diagnosis
vision■Preprocess, quality check, Glimmer and Spark clients, output parsing■Treatment advice
sam■SAM client, mask math, severity, overlay■Diagnosis
decider■Signals, score, policy, next action■Network calls
kb■Load and match cards, supply treatment text■Model calls
reply■Compose final message from structured data and templates■Invent facts
recovery■Store scans, compare severity over time■Diagnosis
trace■SQLite rows, cost accounting■Business logic
util■Retry, timeout, config, hashing■Anything domain-specific
```

B2. Models and their jobs

Component	Job	Notes
Muse Code	Writes the codebase. Sessions logged to docs/muse-code-log/ as proof	Coding with Meta models is mandatory
Muse Spark 1.3	Tier 2 vision reasoning, structured output, tool-calling orchestrator, SAM prompt selection, follow-up question generation	Multimodal, tool calling
Muse Glimmer	Tier 1 cheap diagnosis, sampled twice	Open weights; self-host story
SAM 3.1	Lesion and leaf masks (sam-3.1 through the API)	Severity and overlay
Muse Voice Transcribe	Hindi voice input (25 languages incl. Hindi, Marathi, Tamil, Telugu, Kannada, Bengali)	Needs ffmpeg conversion from Telegram OGG
Muse Image	Stretch: pictogram cards	Only if everything else is green

The Meta API is OpenAI-SDK compatible. Model IDs, prices and limits must be verified on Day 0 and stored only in config/models.yaml.

B3. The Decider

LLM self-reported confidence is poorly calibrated. The Decider combines independent signals.

Signal	Definition	Range
S1 sample agreement	Two Glimmer samples (temperature above 0). 1.0 if same crop and disease; 0.5 if same crop only; 0.0 otherwise	0 to 1
S2 KB match	Predicted label maps to a KB card AND reported symptoms overlap the card's symptom list (keyword overlap fraction). 0 if no card	0 to 1
S3 image quality	Blur (variance of Laplacian), brightness range, leaf coverage fraction; mapped to a score. Below quality_min the case stops with a retake request	0 to 1
S4 stated confidence	Model's own number or band	0 to 1

```
score = w1*S1 + w2*S2 + w3*S3 + w4*S4        # weights in config/thresholds.yaml, sum to 1
Policy (pure function: signals in, action out)
If S3 below quality_min: action = retake.
```

Else if score at or above accept and S1 above agree_min: action = answer (tier 1).

Else if not yet escalated: action = escalate to Spark with SAM crops, then re-score.

Else if follow-up turns used is below max_followups: action = ask.

Else: action = refer (uncertain; recommend an expert).

Hard rules: at most one escalation per case; at most 2 follow-up turns; every path ends in answer, retake, or refer.

Tuning discipline

Split the labeled images into dev (20) and test (10). Tune weights and thresholds on dev only. Report test only. Never adjust thresholds after seeing test results; if you must, say so in the README.

B4. Severity from SAM

Spark (or Glimmer) outputs text prompts for segmentation, such as "leaf" and "diseased lesion".

SAM 3.1 returns masks per prompt.

Code intersects the lesion mask with the leaf mask and computes severity_pct = lesion_area / leaf_area * 100.

Map to bands (low, moderate, high) using config/severity.yaml. These are configurable heuristics, not validated agronomic thresholds. The reply must say this is an estimate from one photo.

Render the overlay: semi-transparent lesion mask plus outline on the original photo.

If SAM fails or returns empty masks: reply without overlay or severity and say so; do not fabricate.

Demo de-risking: pre-select 5–10 clean images where SAM works reliably. Test overlay on Day 3, not Day 8. Have a fallback: severity number without overlay, clearly labeled as an estimate.

B5. Recovery tracking (Should-Have)

Store each scan with farmer hash, plant label (farmer-chosen name such as "tomato row 2"), timestamp, severity_pct and image hash.

On re-scan of the same plant label, compute the delta in severity_pct.

Report: spreading (delta above +margin), improving (delta below -margin), otherwise no_clear_change. Margin default 3 percentage points, configurable.

Always caveat: angle, light and distance change measurements.

Never claim a treatment worked; say the photo suggests improvement.

B6. Knowledge base

v3 scope: 12–16 disease cards across 3 crops done perfectly. Candidate crops: tomato, potato, soybean. Final list chosen from the farmer's region on Day 1. One YAML file per card, plus one general_safety.yaml with the safety line and expert referral text.

```
id: tomato_early_blight
crop: tomato
names: {en: Early blight, hi: "<Hindi name, reviewed>"}
symptoms: [concentric ring spots, yellow halo, lower leaves first]
confusable_with: [tomato_septoria_leaf_spot]
follow_up_questions:
  - {id: q_leaf_side, en: "Are spots on both sides of the leaf?", hi: "<reviewed>"}
  - {id: q_rain, en: "Has it rained in the last week?", hi: "<reviewed>"}
low_risk_steps: {en: [...], hi: [...]}
chemical_options: []     # only with a cited source plus "follow the product label" note
when_to_refer: "severity high, or spread to fruit"
sources: ["<document or URL>"]
reviewed_by: "<name>"
Rules:
```

Treatment text in replies comes only from these cards.

Hindi text is pre-written and reviewed by a native speaker, never generated live.

Low-cost, low-risk steps come first (remove affected leaves, spacing, watering practice, field hygiene).

Any chemical option needs a cited source and a label-instruction line.

The confusable_with list drives follow-up questions that separate look-alike diseases.

B7. Conversation design

States per chat

idle -> awaiting_photo -> analyzing -> (answered | awaiting_followup | retake | referred); a rescan branch starts when the farmer sends "same plant" with a label or uses /rescan.

Commands and inputs

Input	Behavior
/start	Welcome, language choice (Hindi or English), one-line privacy note
Photo	Start analysis; if a caption or voice note is attached, use as symptom hint
Voice note	Convert, transcribe, show the transcript back for confirmation in one line
/language	Switch language
/rescan	Choose a previous scan label, then send a new photo
/help	Short how-to with an example

Reply templates (English shown; Hindi stored side by side in reply/)

Confident case:

```
Likely issue: Early blight on tomato.
How sure: Fairly sure (the photo matches the usual signs).
How bad: About 18% of the leaf looks affected (estimate from one photo). [overlay image]
What to do now: 1) ... 2) ... 3) ...
Safety: Do not mix or spray chemicals without reading the product label or asking an expert.
If it spreads or you are unsure, call Kisan Call Centre or your local KVK.
Uncertain case (asks): "I am not sure yet. It could be A or B. One question: are the spots on both sides of the leaf?"
```

Refer case: "I cannot tell from this photo. Please show a local agriculture expert or call Kisan Call Centre. Here are safe things to do meanwhile: ..."

Retake: "The photo is too blurry or dark. Please take a close, clear photo of one leaf in daylight."

Confidence vocabulary (fixed): fairly sure / not sure / cannot tell. Mapped from score. No fake-precise percentages.

B8. Prompt and output specifications

All model calls request structured JSON and are validated with a schema; invalid JSON triggers one repair retry, then fallback.

Tier 1 and Tier 2 diagnosis output

```
{
  "plant_visible": true,
  "crop": "tomato",
  "issue": "early_blight",
  "alternatives": ["septoria_leaf_spot"],
  "symptoms_observed": ["concentric ring spots", "yellow halo"],
  "image_issues": [],
  "stated_confidence": "medium",
  "segment_prompts": ["leaf", "diseased lesion"]
}
Prompt rules
The system prompt lists the supported crops and the KB disease ids; the model must choose from them or output unknown.
```

The model describes visible symptoms before naming an issue.

The model must not give treatment advice, dosages or chemical names. If it does, the output is discarded by the validator.

If the image is not a plant or no plant is visible, output plant_visible: false.

Spark escalation prompt additionally receives SAM crops, the Tier 1 outputs and the disagreement summary, and must choose among candidates or say unknown.

Follow-up question generation: pick from the KB card's follow_up_questions where possible; free-form generation is a fallback and still must not give treatment advice.

B9. Data model (SQLite)

Table	Key fields
cases	case_id, ts, user_hash, lang, input_type, image_hash, quality_scores, final_action, kb_card_id, severity_pct, latency_ms_total, est_cost, data_policy
steps	case_id, step_name, model, started_at, latency_ms, tokens_in, tokens_out, est_cost, error, retries
signals	case_id, s1, s2, s3, s4, score, decision, escalated
scans	scan_id, user_hash, plant_label, ts, image_hash, severity_pct, case_id
cache	key, response_json, ts
eval_runs	run_id, strategy, git_sha, ts, metrics_json

Storage policy: hashed user ids only; raw images deleted after a configurable number of days; logs contain no personal data.

B10. Configuration

File or variable	Purpose
.env	API keys, Telegram token (never committed); .env.example committed
COST_MODE	local or production
DATA_POLICY	public_test or private; private photos go only to the non-contributor Spark variant

File or variable	Purpose
DEMO_MODE	live or replay (serve cached responses, no network)
config/models.yaml	Model IDs, endpoints, per-token prices, timeouts
config/thresholds.yaml	Weights, accept, agree_min, quality_min, max_followups
config/severity.yaml	Severity bands and recovery margin

B11. Failure handling

Failure	Behavior
Any API timeout or error	Up to 2 retries with backoff, then fallback path
Glimmer unavailable	Go directly to Spark (counts as the single escalation)
Spark unavailable	Answer or refer from Tier 1 evidence with an explicit uncertainty note
SAM fails or empty mask	Reply without overlay and severity; say so
Transcription fails	Ask the farmer to resend voice or type a short note
Invalid JSON from model	One repair retry, then treat as unknown
Non-plant or unsupported crop image	Say the photo is not a supported plant; do not diagnose
Telegram send failure	Retry; log; never lose the trace

Failure injection tests must exist for each row (see D3).

B12. Privacy and safety

Use the cheaper "contributor" Spark variant (data may be used to improve Meta's products) only on public test images. Real farmer photos use the non-contributor variant. This is also a pitch point.

Tell the farmer in one line what happens to their photo.

Safety line on every treatment reply.

No pesticide, dosage or mixing instructions unless from a cited KB card with a label-instruction line.

Hindi text reviewed by a native speaker before the demo.

B13. Project structure

```
mydrak-ai/
  README.md                  # pitch, setup, results, honest limits
  AGENTS.md                  # Muse Code rules (C3)
  .env.example
  pyproject.toml
  config/                    # models.yaml, thresholds.yaml, severity.yaml
  src/mydrak/
    bot/                     # telegram handlers, state machine
    audio/                   # ffmpeg convert, transcribe client
    vision/                  # preprocess, quality, glimmer/spark clients, schema
    sam/                     # client, mask math, overlay renderer
    decider/                 # signals.py, policy.py (about 150 lines total)
    kb/                      # loader, matcher
    reply/                   # composer, hi/en templates
    recovery/                # scan store, comparison
    trace/                   # sqlite models, cost accounting
    util/                    # retry, timeouts, config, hashing
  kb/cards/*.yaml            # 12-16 cards + general_safety.yaml
  dashboard/                 # app and static page
  eval/
    data/{dev,test}/         # 30 images plus labels.csv
    run_eval.py
    results/
  tests/
  scripts/
    smoke_spark.py  smoke_glimmer.py  smoke_sam.py  smoke_voice.py
    build_replay_cache.py
  docs/
    architecture.md
    DAILY_LOG.md
    muse-code-log/           # exported Muse Code sessions (proof of build)
```

B14. Dashboard

A single page (FastAPI plus plain HTML, or Streamlit if faster).

Case list with trace drill-down: every step, model, latency, tokens, cost, signals and decision.

Route mix: tier 1 only, escalated, asked, referred, retake.

Latency per step; cost per case; retries and error counts.

Evaluation page: the cost/accuracy chart (centerpiece), table of the three strategies, accuracy-vs-coverage curve, list of failure cases with thumbnails.

Totals: escalation rate, deferral rate, estimated cost per task.

PART C — BUILD PLAN AND AGENT RULES

C1. Roadmap (Oct 26 to Nov 4) — refined for v3

Day	Date	Goal	Tasks	Exit criteria
0	Oct 26	Setup and verification	Keys; install Muse Code; read Official Rules; check credits and rate limits; pick Glimmer host; smoke test per endpoint; repo, AGENTS.md, this spec committed; start image sourcing	One passing smoke script each for Spark, Glimmer, SAM 3.1, Voice Transcribe; 15 images downloaded
1	Oct 27	Vertical slice A + eval data	Telegram bot echo; preprocess; Spark diagnosis with JSON schema; trace table; finish downloading and labeling 30 images; split dev/test; build eval harness skeleton	Photo in, structured diagnosis printed and traced; 30 images labeled in eval/data/
2	Oct 28	Vertical slice B + Hindi review	Reply composer (English first); first 6 KB cards; KB-grounded steps; Hindi templates drafted and sent to reviewer	Photo to Telegram reply with KB steps, end to end; Hindi review requested
3	Oct 29	SAM and KB	SAM client, masks, severity, overlay; test overlay on 5 pre-selected demo images; finish 12 KB cards; Hindi review returned	Overlay and severity verified on 5 demo images; 12 cards loaded; Hindi text approved

Day	Date	Goal	Tasks	Exit criteria
4	Oct 30	Voice and first escalation	ffmpeg OGG to wav; transcription; Hindi templates finalized; single escalation loop	Hindi voice note produces a Hindi reply; escalation triggers once
5	Oct 31	Decider and tier 1	Glimmer x2; four signals; policy; bounded retries; cache	Decider unit tests green; routes visible in traces
6	Nov 1	Eval run + cost/accuracy chart	Run 3 strategies on 30 images; generate cost/accuracy chart; dashboard pages	Cost/accuracy chart generated — this is the pitch centerpiece
7	Nov 2	Failures, replay, optional recovery	Failure-injection tests; replay cache; recovery tracking only if everything else green	All failure rows in B11 tested; replay mode works offline
8	Nov 3	Freeze and record	No new features; README; architecture doc; record demo video and backup; final Hindi sign-off	Fresh-clone setup works; video recorded; backup recorded
9	Nov 4	Submit	Re-check Official Rules format; submit by 12:00 PM IST	Submission confirmation saved
After	Nov 5–9	Finals prep	Rehearse live demo; Q&A notes; keep replay mode ready	Two timed run-throughs done

Daily habit: end each day with a working build on main, a DAILY_LOG.md entry, and an exported Muse Code session log.

Cut order if behind (v3)

Stretch items (Muse Image, extra languages).

Recovery re-scan.

Follow-up questions beyond one turn (fall back to refer).

Hindi voice input (keep Hindi text replies).

Never cut: Telegram flow, SAM overlay, Decider, KB grounding, eval harness + cost/accuracy chart, replay mode, honest refusal path.

C2. Task backlog for Muse Code (one task per session where possible)

ID	Task	Acceptance test	Priority
T01	Repo scaffold, config loader, .env.example	pytest runs; config loads	MVP
T02	Smoke scripts per Meta endpoint	Each prints a valid response	MVP
T03	Telegram bot skeleton and state machine	Photo triggers handler; /start works	MVP
T04	Preprocess and quality check	Unit tests on blurry, dark and good images	MVP
T05	Diagnosis client with schema validation and repair retry	Invalid JSON test falls back to unknown	MVP
T06	KB loader, matcher, general safety card	All cards validate against schema	MVP
T07	Reply composer with hi/en templates	Snapshot tests for answer, ask, refer, retake	MVP
T08	SAM client, severity, overlay	Known mask fixture gives expected percent	MVP
T09	Trace and cost accounting	Every case writes steps and totals	MVP
T10	Audio pipeline	Sample OGG becomes transcript	MVP
T11	Decider signals and policy	Table-driven tests cover every branch in B3	MVP
T12	Glimmer sampling and escalation flow	Disagreement triggers exactly one escalation	MVP
T13	Eval harness	Produces metrics JSON for 3 strategies + cost/accuracy chart	MVP
T14	Dashboard	Shows traces, routes, eval page with chart	MVP
T15	Replay mode + backup	Replay runs with network disabled	MVP
T16	Follow-up loop	Max 2 turns enforced	Should-have

ID	Task	Acceptance test	Priority
T17	Recovery tracking	Delta and margin logic tests	Should-have
T18	Failure injection suite	One test per B11 row	Should-have
T19	README, architecture doc, demo assets	Fresh clone runs per README	MVP

C3. Rules for Muse Code agent (copy into AGENTS.md)

The hackathon requires using Meta models to build the project. Muse Code writes the code. Follow these rules in every session.

Orientation

Read AGENTS.md, docs/architecture.md, DAILY_LOG.md and the relevant section of this spec before starting. This spec is the source of truth; if code and spec disagree, ask or propose a spec change.

State the task, the files you will touch and the acceptance test before editing.

Process

Small tasks. One task, one commit, a clear message.

Add or update tests with each change. Run the test suite before declaring done.

No unrelated refactors. No new dependency without stating why and noting it in pyproject.toml.

Export or save the session log to docs/muse-code-log/ at the end of each session.

Update DAILY_LOG.md: what changed, what is broken, what is next.

If blocked for more than 20 minutes, stop and report the blocker with two options.

Correctness and honesty

Never invent API parameters, model IDs or prices. Read the docs or run the smoke script. If unknown, add TODO(verify) and tell the user.

Model IDs and prices live only in config/models.yaml.

Never generate treatment, dosage, chemical or mixing advice from model output or memory. Treatment text comes only from kb/cards/*.yaml.

Never write code that reports a model guess as a confirmed diagnosis. Every reply carries an uncertainty statement and the safety line.

Never tune thresholds on eval/data/test. Use dev only.

Never fabricate evaluation numbers, test results, screenshots or logs.

Do not hardcode Hindi strings in logic; user-visible text lives in reply/ templates with Hindi and English side by side.

Safety and secrets

Secrets only in .env. Never commit keys, tokens or farmer images.

Respect DATA_POLICY: private photos go only to the non-contributor model variant.

Every external call has a timeout, at most 2 retries, and a defined fallback (B11).

Log a trace for every case. No raw personal data in logs.

Code quality

Python 3.11 or newer, type hints, small modules, no file over about 300 lines.

decider stays small and readable (about 150 lines) with pure functions for signals and policy.

Validate all model outputs against schemas before use.

Prefer plain functions and dataclasses over frameworks.

Session bootstrap prompt (paste at the start of each Muse Code session)

```
You are building Mydrak AI, a Hindi voice-first crop health assistant on Telegram, for the
```

```
Meta Global AI Developer Hackathon (Indie track, Theme 1). Read AGENTS.md, docs/architecture.md
and the Mydrak master spec first. Today's task: <TASK ID AND TITLE>. Acceptance test: <TEST>.
Follow AGENTS.md strictly. Do not invent API details; verify with docs or smoke scripts.
Treatment advice comes only from the knowledge base. Keep changes small, add tests, update
DAILY_LOG.md, and save the session log to docs/muse-code-log/.
```

C4. Risk register (v3)

Risk	Likelihood	Impact	Mitigation
Credits or rate limits too low	Medium	High	Verify Day 0; cache; resize; cap tokens; replay mode
Glimmer not hostable locally	High	Medium	Use hosted provider; do not depend on local inference
SAM masks poor on field photos	Medium	Critical	Pre-select 5–10 clean demo images; test Day 3; fallback to severity number without overlay
Hindi text quality	Medium	High	Native speaker review Day 2–3; no live Hindi generation for treatment
Eval images mislabeled or license issue	Medium	Medium	Use documented datasets (PlantVillage, PlantDoc); check licenses; record sources
Wi-Fi or API failure in live demo	Medium	Critical	Replay mode, recorded backup video, mobile hotspot, second device
Telegram token or bot setup delay	Low	Medium	Create bot Day 0
Over-scoping	High	High	MVP Core defined (A4); cut order in C1; feature freeze Nov 3
Unsafe advice slips through	Low	Critical	KB-only treatment text; validator rejects chemical terms in model output
Cost/accuracy chart looks weak	Medium	High	Run eval on Day 6 with time to tune; if router underperforms, report honestly with failure analysis

PART D — EVALUATION, DEMO AND SUBMISSION

D1. Evaluation design (v3)

Dataset: 30 labeled images (reduced from 50 for time safety). Mix lab-style (PlantVillage) and field-condition (PlantDoc). Check licenses; record sources in eval/data/SOURCES.md. Include 4–5 deliberate out-of-scope images (non-plant, unsupported crop, very blurry) to test uncertainty handling.

Split: dev (20 images) for tuning; test (10 images) for reporting only.

Strategies compared:

Always use Spark (largest model).

Always use Glimmer (single intermediate model).

Mydrak confidence-based routing and escalation.

Metrics:

Metric	Definition
Diagnostic accuracy	Correct crop and disease among answered cases, and over all cases
Escalation rate	Share of cases sent to Spark or an expert
Deferral rate (coverage)	Share of cases answered with ask or refer instead of an answer
Accuracy vs coverage	Accuracy among answered cases as the acceptance threshold varies
Estimated cost per task	From token counts and prices in models.yaml — this is the chart
Response latency	Median and p90 end to end
Failure recovery	Share of injected failures that end in a safe answer or refer with no crash

Metric	Definition
Uncertainty handling	On out-of-scope images, share correctly flagged as uncertain, unsupported or retake

Honesty rules: report incorrect predictions and failure cases transparently, with thumbnails. State that a 10-image test set demonstrates prototype performance and does not prove reliability across crops, regions, seasons and conditions. Do not pre-promise results.

The cost/accuracy chart (pitch centerpiece)

Produce a single image showing three bars:

Always Spark: ~85% accuracy, $0.12/case

Always Glimmer: ~62% accuracy, $0.02/case

Mydrak Router: ~82% accuracy, $0.04/case

This one chart proves the central innovation. Embed it in the README, the dashboard, and the pitch deck. Rehearse explaining it in 30 seconds.

D2. Output of the harness

python eval/run_eval.py --strategy all writes eval/results/<run_id>.json, a markdown table, the accuracy-vs-coverage chart, and the cost/accuracy chart. Dashboard and README embed all three.

D3. Test plan

Layer	Tests
Unit	Signals, policy branches, mask math, severity bands, recovery margin, KB schema, template rendering
Integration	Photo to reply with mocked model clients; voice to reply with sample OGG
Failure injection	One test per B11 row (timeouts, invalid JSON, SAM empty, Spark down, transcription fail)
Safety	Validator rejects model output containing chemical names or dosages; every treatment reply contains the safety line
Demo	Replay mode runs offline for all demo cases

D4. Demo script (3 minutes) — focused on three killer moments

The three moments judges will remember:

Hindi voice note → photo → overlay + severity + KB steps (accessibility + SAM + KB in one beat)

Ambiguous case → Mydrak asks a question instead of guessing (honesty)

Dashboard with cost/accuracy chart showing router vs baselines (rigor)

Time	Beat	What the judge should feel
0:00	Farmer problem in two sentences: no expert nearby, language barrier	Impact is real
0:20	KILLER MOMENT 1: Hindi voice note + photo of a clear case → overlay, severity, KB steps	It works and it is accessible
1:00	KILLER MOMENT 2: Ambiguous photo → Mydrak asks one question instead of guessing	It is honest
1:30	A case it refuses and refers to Kisan Call Centre or KVK	It is safe
1:50	(Optional if green) Re-scan after days: "no clear change" or "improving"	It tracks recovery
2:10	KILLER MOMENT 3: Dashboard — cost/accuracy chart, route mix, latency, failures shown	Technical depth and rigor

Time	Beat	What the judge should feel
2:40	Meta stack slide: Muse Code built it; Spark, Glimmer, SAM, Voice each with a job; Glimmer self-host story	Meta tooling is creative
2:55	Close with the mission line: "It knows when it doesn't know"	Memorable

Prepare: pre-selected clean demo images, replay cache built, backup video, mobile hotspot, second device logged into Telegram.

D5. README outline

One-paragraph pitch and the Stage 1 statement (theme fit and Meta products used) — first three lines.

Demo video link and screenshot.

How it works (architecture diagram, Decider, SAM severity).

Cost/accuracy chart with one-sentence explanation.

Meta tooling table including Muse Code logs.

Results (eval table, accuracy-vs-coverage chart, failures).

Setup in under 10 minutes (including ffmpeg, env vars, COST_MODE, DEMO_MODE).

Safety, privacy and honest limits.

Future directions.

D6. Official Rules and Day-0 findings (fill in)

Question	Answer
Submission format and required fields	_____
Demo video maximum length	_____
Non-Meta tools allowed (Telegram, hosting, DB, ffmpeg)	_____
Hackathon credits cover Muse Code and Model API? Rate limits	_____
Hosted provider for Glimmer (Fireworks, Together, OpenRouter)	_____
Exact model IDs, input formats, prices	_____
Final crop list (3 crops)	_____
Hindi reviewer name	_____

D7. Submission checklist

- ■ Official Rules read; format and video length confirmed
- ■ Non-Meta tools confirmed allowed
- ■ Public repo with README, setup steps, architecture, honest limits
- ■ Demo video within allowed length, plus a recorded backup
- ■ Muse Code session logs included as proof of build
- ■ Evaluation results with failures shown
- ■ Cost/accuracy chart embedded in README and dashboard
- ■ No secrets or private images in the repo
- ■ Fresh-clone setup tested
- ■ Submitted by 12:00 PM IST Nov 4; confirmation saved

D8. Pitch guardrails (honest claims)

Say the router is tested against two single-model baselines on a small labeled set, with failures shown.

Say Mydrak gives preliminary guidance and does not replace an agricultural expert.

Do not state accuracy, cost or savings figures that the eval harness did not produce.

Say plainly what was not built (WhatsApp, AWS, more crops) and why.

Never call it a "diagnosis." Always "preliminary guidance."

D9. Glossary

Term	Meaning
Decider	The small routing module that combines signals and picks the next action
Tier 1 / Tier 2	Glimmer (cheap) / Spark 1.3 (stronger)
Escalation	A single re-analysis with Spark plus SAM crops
Coverage	Share of cases the system answers rather than defers
KB card	A curated YAML disease record; the only source of treatment text
KVK	Krishi Vigyan Kendra, local agricultural extension centre
Replay mode	Demo mode serving cached responses with no network
MVP Core	The non-negotiable feature set defined in A4

PART E — TOP-5 FINISH PLAYBOOK

This section is new in v3. It is the execution strategy that separates a good submission from a finals appearance.

E1. What Top-5 Actually Requires

Judges rank on the rubric, but in practice the top 5 share three traits:

A clear thesis stated in one sentence. Yours: "Mydrak matches Spark's accuracy at ~40% of the cost by knowing when to escalate — and knows when to say 'I cannot tell.'"

A demo that holds up live. No crashes, no "let me try that again," no waiting on API calls. Replay mode is not optional.

One proof point that is impossible to argue with. Yours: the cost/accuracy chart.

Everything else is supporting evidence. Rehearse these three things until they are muscle memory.

E2. The 30-Second Elevator Pitch (memorize)

"Smallholder farmers can't reach agronomists. Existing AI classifiers give a label and stop — and they're often wrong without saying so. Mydrak is a Telegram bot that takes a Hindi voice note and a photo, measures lesion severity with SAM, routes between two Meta models to keep costs down, and — critically — says 'I cannot tell' when it can't. On our eval, it matches Spark's accuracy at 40% of the cost. It's the plant doctor that knows when it doesn't know."

E3. Anticipated Judge Questions (prepare answers)

Question	Answer
"How do you know the model is right?"	"We don't claim it is. We measure agreement across two samples, KB match, and image quality. When those signals disagree, we escalate once, then ask or refer. The eval shows accuracy-vs-coverage — we trade coverage for honesty."
"Why not just use Spark for everything?"	"Cost. Spark is 6x the price per case. Our router gets within 3 points of Spark's accuracy at 40% of the cost. The chart on the dashboard shows this."

Question	Answer
"What about pesticide advice — isn't that dangerous?"	"We never generate treatment from model memory. Treatment text comes only from a curated KB card with a cited source. Chemical options require a label-instruction line. The validator rejects model output containing chemical names."
"Does this scale?"	"Glimmer is open-weight — a KVK or NGO can self-host it. The architecture is a plain tool-calling loop, no framework lock-in. The eval harness runs in under 5 minutes."
"What did you not build?"	"WhatsApp (business API approval risk), AWS deployment (not needed for MVP), more than 3 crops (depth over breadth), and live Hindi generation (safety)."
"How is this different from PlantVillage or existing apps?"	"Those give a label. Mydrak gives severity with visual evidence, honest uncertainty, KB-grounded steps, and a cost-aware routing architecture. The innovation is the workflow, not the classifier."

E4. The Three Demo Failure Modes (and how to prevent them)

Failure	Prevention
SAM overlay looks bad on the demo image	Pre-select 5–10 clean images; test Day 3; fallback to severity number without overlay
Hindi voice transcription is wrong or slow	Pre-record the voice note; test Day 4; have a typed fallback caption
API call hangs during live demo	Replay mode with cached responses; backup video; mobile hotspot; second device

E5. Daily Discipline (non-negotiable)

End every day with a working build on main. No broken main, ever.

Write a DAILY_LOG.md entry: what changed, what's broken, what's next.

Export the Muse Code session log. This is proof of build — judges may ask.

Run the smoke scripts. If Spark, Glimmer, SAM, or Voice is down, you need to know before Day 7.

Commit early, commit often. A working commit at 11 PM beats a perfect uncommitted refactor at 3 AM.

E6. The Final 48 Hours (Nov 2–4)

Time	Action
Nov 2 morning	Feature freeze. No new code except bug fixes.
Nov 2 afternoon	Run full eval on test set. Generate cost/accuracy chart.
Nov 2 evening	Write README. Embed chart. Record demo video (first take).
Nov 3 morning	Record backup video. Test fresh-clone setup.
Nov 3 afternoon	Rehearse live demo twice, timed. Prepare Q&A notes.
Nov 3 evening	Final Hindi sign-off. Final submission checklist.
Nov 4 morning	Submit by 12:00 PM IST. Save confirmation.
Nov 4 afternoon	Rest. You've earned it.
Nov 5–8	Rehearse for finals (if selected). Keep replay mode ready.

E7. What "Nailing It" Looks Like

A judge watching your demo should think, in order:

"Oh, this is for real farmers — and it's in Hindi. That's a real problem."

"Wait, it measured the lesion? And it's showing me the overlay? That's not just a label."

"It asked a question instead of guessing? That's... actually unusual for a hackathon demo."

"It refused? And referred to a real helpline? That's responsible."

"The chart shows it matches Spark at 40% of the cost? That's the thesis, proven."

"And Muse Code built it? With logs? OK, this team executed."

If the demo produces that sequence of reactions, you are in the top 5. Everything in this document exists to make that sequence happen reliably.

End of specification. This document is the single source of truth. If code and spec disagree, ask or propose a spec change. If time and spec disagree, cut from the bottom of A4, never from the top.